

Whack-a-Mole: Technical Report

To run the program execute the following commands in the terminal:

```
./build.sh
```

```
java -cp bin com.whackamole.Main
```

A) OOP Design and Justification:

Below is the class hierarchy and separation of responsibilities implemented in the program:-

- `com.whackamole.model:`
 - `HoleOccupant`- the abstract class for any object that can appear from a hole. It contains shared state and common methods
 - `Mole`(standard target), `Bomb`(penalty object) and `BonusMole`(special, high value target) are the concrete subclasses.
- `com.whackamole.ui:`
 - `GameBoard`: The main GUI window. Creates and arranges the score display, timer and the restart button. Also displays the grid of `HoleButtons`. It owns the `GameEngine` thread, manages score and receives UI updates from the engine via `SwingUtilities.invokeLater`. It invokes the polymorphic `whack()` on `HoleOccupants`.
 - `HoleButton`: UI representation of a single hole in the grid
 - `ImageUtils`-used for loading and scaling icons
- `com.whackamole.engine:`
 - `GameEngine`-controls the timing and game logic on a dedicated thread. Responsible for regularly decrementing the timer, controls lifespan of `HoleOccupants`, sends UI updates to `SwingUtilities.invokeLater`
- `com.whackamole.persistence:`
 - `PlayerScore`-contains a player's name and final score. It implements `Serializable` so that the list of scores are written directly to a file
 - `HighScoreManager`-Handles all file I/O for scores using `ObjectOutputStream` and `ObjectInputStream`. It loads an `ArrayList<PlayerScore>` from `scores.dat` and saves updated lists.
- `com.whackamole.exceptions:`
 - `HighScoreException`(checked): custom checked exception to handle persistence errors.
 - `InvalidGameStateException`(unchecked): runtime exception signaling programmer errors.

`HoleOccupant` encapsulates common state (visibility, lifespan) and a common contract (`whack()`, `getIcon()`). Concrete classes (`Mole`, `Bomb`, `BonusMole`) implement behavior differences (score change, special effects). This avoids instance-of checks in event handlers since the UI simply calls `occupant.whack()` and acts on the returned score.

`GameBoard` has-a `GameEngine` and an array of `HoleButtons`. `HoleButton` has a `HoleOccupant`.

`ImageUtils` centralizes image downloading and ensures it is scaled accurately. This ensures consistent icon sizing.

B) Concurrent strategy

- GameEngine implements Runnable and runs on a dedicated thread (new Thread(engine)).
- The GUI runs on the Swing Event Dispatch Thread (EDT). All Swing updates (label changes, icons on JButtons) occur on the EDT.
- All changes to Swing components originating from GameEngine are dispatched to EDT using SwingUtilities.invokeLater(). Single-thread rule for swing is obeyed and race conditions, repaint glitches, and concurrency exceptions. are prevented
- HoleButton contains a small synchronized section around its occupant reference to read/write the HoleOccupant safely. When the engine chooses a hole to spawn into, it checks target.getOccupant() under a short synchronization block and only enqueues the UI update to set the occupant if the hole is still empty. This prevents subtle races where the UI or another engine tick might concurrently modify the same hole.
- GameEngine.run() contains Thread.sleep(...) for pacing. The sleep call is inside a try/catch(InterruptedException) block that does not swallow interrupt. it sets the running flag to false (or exits the loop) and marshals a final UI update to show time=0 and clear the board. The window-closing handler calls gameThread.interrupt() and joins it. This ensures a clean shut down.
- UI event handlers call board.handleWhack(index) which perform the minimal model update and schedule heavy work off the EDT. this ensures the UI is responsive.
- The timing logic in GameEngine is isolated so that UI is responsive.

C) Custom Exceptions:

- HighScoreException (checked)
 - Extends: Exception (checked)
 - Purpose: Wraps I/O-related issues during serialization/deserialization (e.g., IOException, ClassNotFoundException) and forces callers to handle persistence errors explicitly.
 - A checked exception ensures the code that calls loadScores() or saveScores() must either handle the error (show a friendly alert, fall back to empty list) or propagate it deliberately
 - For instance, HighScoreManager.loadScores() catches low-level exceptions and throws new HighScoreException("Failed to load scores", e). The UI catches this and displays a warning dialog while continuing.
- InvalidGameStateException(unchecked):
 - Extends: RuntimeException (unchecked)
 - Purpose: Signals an unrecoverable logic bug (programmer error), such as an invariant violation that "should never happen" — e.g., code that attempts to spawn into a hole that is unexpectedly occupied despite prior checks.
 - Unchecked exceptions are appropriate for unrecoverable, unexpected states that indicate a bug in the program logic. They should not be used to control normal flow (the engine now *skips* occupied holes instead of throwing).

